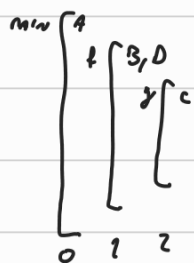


# 1) SCOPING STATICO / DINAMICO

↳ STATICO

↳ DINAMICO

- SUDDIVIDERE IL PROGRAMMA SECONDO LE VARIABILI DICHIARATE IN QUALCUNA SEZIONE  
 E SERVARE ANCHE IL LIVELLO (SD)



$$SD(main) = 0$$

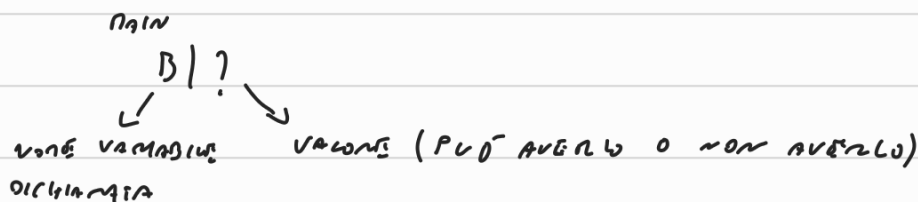
$$SD(f) = 1$$

$$SD(g) = 2$$

↳ ORDINE DI CHIAMATA  
 $main \rightarrow f() \rightarrow g()$

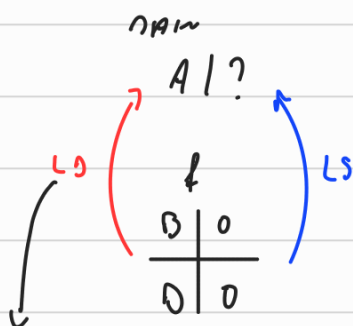
## SCOPING STATICO

PARTE DA MAIN E FACILIO



VADO ALLA FUNZIONE CHIAMATA SUBITO DOPO

$main \rightarrow f$



CALCOLO IL LINK STATICO

$$LS = SD(chiamante) - SD(chiamata) + 1$$

$$LS = SD(main) - SD(f) + 1 = 0 - 1 + 1 = 0$$

IL LINK DINAMICO PUNTA SEMPRE

ALLE FUNZIONI PRECEDENTI

N.D.

SE  $LS = 0$  LA FRECCIA PUNTA ALLA FUNZIONE

PRECEDENTE, SE  $LS = 1$  PUNTA A 2 FUNZIONI

PRECEDENTI, SE  $LS = 2$  PUNTA A 3 FUNZIONI PRECEDENTI

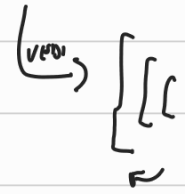
Se devo fare un'operazione es  $x = 7 + 9$  (soprattutto  $x$  e  $y$  locali)

Devo trovare  $N_7$

→ continuo di LS

$$N_7 = SD(\text{funzione attuale}) - SD(\dots)$$

↳ calcolo la prima funzione che possiede la variabile che cerca, tornando indietro di gerarchia



a) suddiviso  $[c, SD, \text{città}]$

b) per ogni passaggio nuovo LS

c) calcolo l'ultima operazione per ogni passaggio

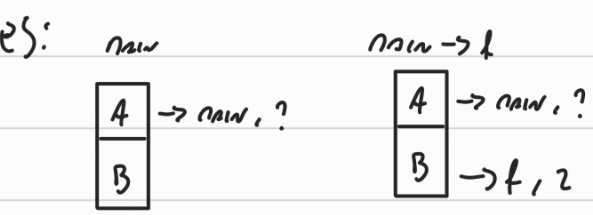
Quando siamo fuori: per ogni punto  $\delta$  la stack viene distrutta

Scopiar dinamico

Però in colonna le variabili



Per proseguire, su ogni step ( $main \rightarrow f, f \rightarrow \dots$ ), siamo accaniti ad ogni variabile in ogni funzione e che valore assume



N.B

2° valore

1° valore

$\boxed{x} \rightarrow f, 2 \rightarrow g, 1$

Quindi la componente  $f, 2$  è stata aggiunta alla SK

Per svolgere un'operazione, bisogna essere utilizzati come più a SK

ES:  $x = y + z = 1 + 2$

|   |                                                 |
|---|-------------------------------------------------|
| b | $\rightarrow$ non ?                             |
| v | $\rightarrow C, 1 \rightarrow D, 7$             |
| w | $\rightarrow B, 3$                              |
| x | $\rightarrow B, \cancel{3}^6 \rightarrow A, 6$  |
| y | $\rightarrow C, \underline{1} \rightarrow A, 0$ |
| z | $\rightarrow D, \underline{2} \rightarrow A, 5$ |

ALLA FINE SCARICO: DANNO TUTTO E LA CRT VIENE DISMONTATA

2) SCOPE STATIC/DYNAMIC / SHALLOW BINDING

VAMABILI... (1)

4 3

}

$p_1 \{$

③

}

VAMADILY... (2)

$$1 \leq k = \dots$$

— — — — —

SCOPE STATICO  $\rightarrow$  USO VARIABILI AMBIENTE (1)

1. Vedo quali cosa risulta il numero di  $f$  e  $f_1$   
e cosa risulta 2-

# 1 SCOPPI DINAMICO

DEEP BINDING  $\rightarrow$  USE VARIABLE ADDRESS (2)

VEDO CHIARI COSA RISULTA IL MURALE DI f e f<sub>1</sub>  
1 E COSA RISULTA 2-

1 SHALLOW BINDING  $\rightarrow$  use variable environment (3)

1. VEROGLI COSA RISULTA IL MEFURAN DI f e f<sub>1</sub>

3. 654 45678 2-

Se si pensa una famiglia (5:4), provo a moltiplicare

P.S: SE IN  $\mathbb{Z} = f_2(1)$  (AND NOT  $\textcircled{2}$ ), POSS. PHENOMENON

LA VAMADILE CHE MI SURRE IN ② PENCHE FACCIAMO UN

PASSO INTERNO É VAZÃO A VÉLOCIDADE DOVE É CHAMADA  $f_1$

### 3) ricorsione

Es. lista = (2, 4, 6)

function (2, 4, 6)

↳ concat (2, function (4, 6)) (6, 12, 18)

↳ concat (4, function (6)) (12, 18)

↳ concat (6, function ()) (18)

↳ ()

□ X → analisi ottimizzata □  
con analisi numero X

→ nell'es. □ X = 3

Modifiche in coda

lista function2 (lista list, list return) {

if (length(list) == 0) then return result;

else return function2 (con(list), concat(result, 3 \* con(list)));

}

lista fun (lista list) {

return function2 (list, ());

}

fun ((2, 4, 6))

↳ function2 ((4, 6), concat ((), 3 \* 2) 6

↳ function2 (6, concat (6, 3 \* 4) (6, 12)

↳ function2 ((6, 12), 3 \* 6) (6, 12, 18)

3. È l'istesso, possono essere  
altre operazioni

#### 4) PROVA DI COMPLETAMENTO

AVVOLGIMENTO CODICI AL POSTO DI (\*) O DI (\*\*)

SCOPE STATICO

es:

... => FUNCTION() => ... = ... | i=0 => FUN() => X=3

... => FUNCTION() => ... = ... | i=2 => FUN() => X=3

SCOPE DINAMICO

es:

... => FUNCTION() => ... = ... | i=0 => FUN() => X=3

... => FUNCTION() => ... = ... | i=2 => FUN() => X=2

#### 5) PASSAGGIO VALORI / RIFERIMENTO

PASSAGGIO VALORI

QUANDO PASSO LA VARIABILE NE PASSO UNA COPIA, IN CASO DI MODIFICHE DELLA VARIABILE, VENGONO MODIFICATE SOLO LE COPIE

PASSAGGIO RIFERIMENTO

QUANDO PASSO LA VARIABILE NE PASSO L'INDIRIZZO DELLA VARIABILE STESSA, QUINDI UN RIFERIMENTO A QUELLA ORIGINALE, IN CASO DI MODIFICHE DELLA VARIABILE VENGONO MODIFICATE IL VALORE DELLA VARIABILE ORIGINALE

6)

$P[\dots]$

$\sigma[\dots]$  memoria

$\dots := \dots$

es:  $P[x \mapsto 3, z := l_x]$

$\sigma[l_x \mapsto 2]$

$z := 5(x + z)$

DEVO TROVARE UN'ALTRA FORMAZIONE NECESSARIA E POI SVOLGO L'OPERAZIONE

es: trovo  $l_x$

valore  $x$   $P \vdash \langle x, \sigma \rangle \rightarrow \langle 3, \sigma \rangle$   $P(x) = 3$

valore  $z$   $P \vdash \langle x + z, \sigma \rangle \rightarrow \langle 3 + z, \sigma \rangle$

valore  $z := \dots$   $P \vdash \langle 5(x + z), \sigma \rangle \rightarrow \langle 5(3 + z), \sigma \rangle$

$P \vdash \langle z := 5(x + z), \sigma \rangle \rightarrow \langle z := 5(3 + z), \sigma \rangle$

PER  $z$  È UN'ALTRA, SOLO CHE IN UNO DEI CONTESTI DI  $x$ , QUINDI PARTIRÒ DAL BASSO CON:

$P \vdash \langle z := 5(3 + z), \sigma \rangle \rightarrow \langle z := 5(3 + 2), \sigma \rangle$

POI SCRIVERÒ:  $P(z) = l_z$  SE CIO' CHE C'È IN  $z$  È IN MEMORIA  
 $\sigma[l_z] = 2$

POI SVOLGO L'OPERAZIONE CON:

$P \vdash \langle 3 + 2, \sigma \rangle \rightarrow \langle 5(5), \sigma \rangle$

$P \vdash \langle 5(3 + 2), \sigma \rangle \rightarrow \langle 5(5), \sigma \rangle$

$P \vdash \langle z := 5(3 + z), \sigma \rangle \rightarrow \langle z := 5(3 + 2), \sigma \rangle$

$P \vdash \langle 5(5), \sigma \rangle \rightarrow \langle 25, \sigma \rangle$

$P \vdash \langle z := 5(5), \sigma \rangle \rightarrow \langle z := 25, \sigma \rangle$

is concluded can:

$$\rho \vdash (\tau := 25, \sigma) \rightarrow \rho[\tau] = l_2$$

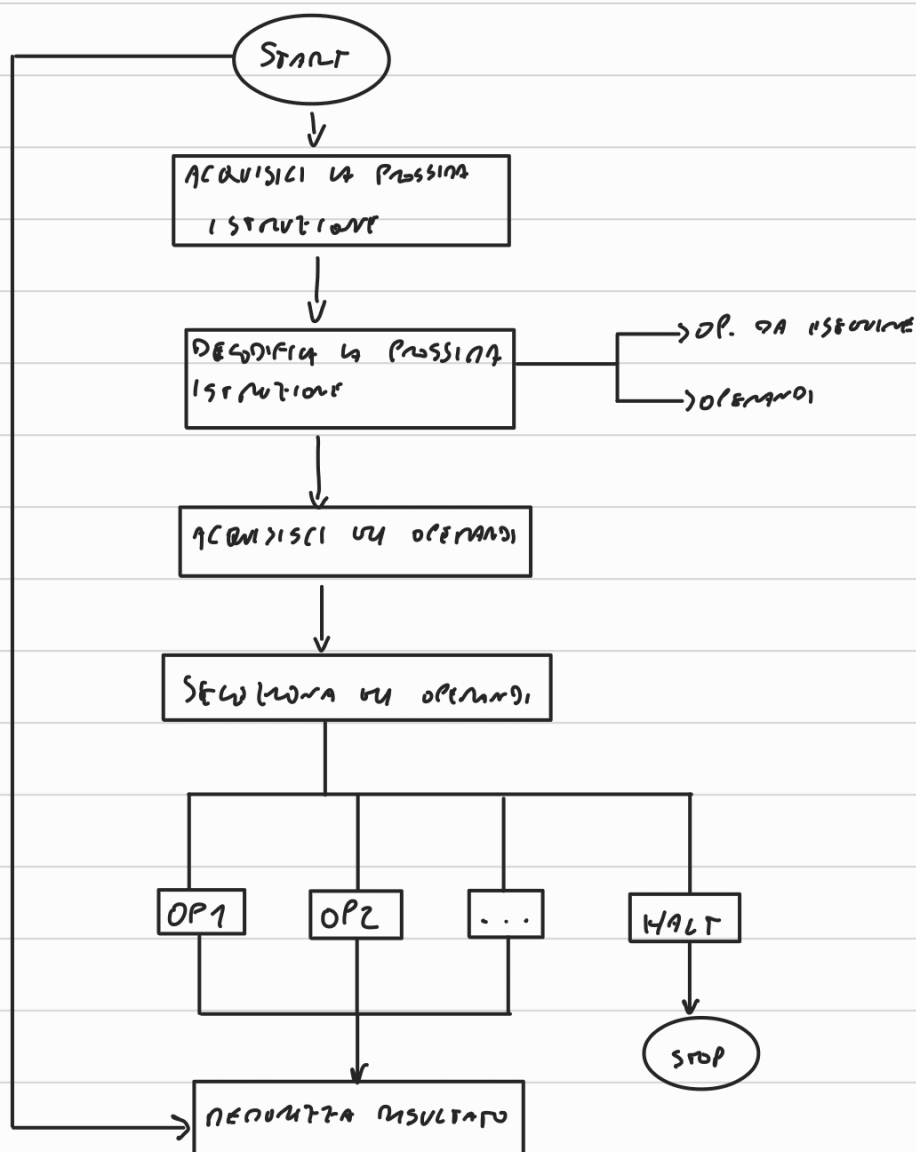
$$\sigma[l_2 \mapsto 25]$$



Un interprete ILL0, sviluppato in L0, è un programma che prende in input un altro programma scritto nel linguaggio L e anche un input da usare con quel programma. L'interprete esegue il programma in L passo dopo passo, utilizzando solo le funzioni e le operazioni disponibili nella macchina astratta di L0, e alla fine restituisce il risultato dell'esecuzione.

$$I^{L_0} : (P_{L_0}^L \times D_{L_0}) \rightarrow D_{L_0}$$

$$[I^{L_0}](P_{L_0}^L, input) = [I^{L_0}](input)$$



Un compilatore è un programma CLL0 che traduce (preservandone semantica e funzionalità) un programma scritto in L in un programma scritto in L0, quindi eseguibile direttamente sulla macchina astratta per L0.

$$C^{L0}: \text{PROG}^L \rightarrow \text{PROG}^{L0}$$

$$[C^{L0}](P^L) = P^{L0} \text{ t.c. } \forall \text{input} \in D, [P^L](\text{input}) = [P^{L0}](\text{input})$$

